

Internet Applications Design and Implementation

Week 8 - Microservices

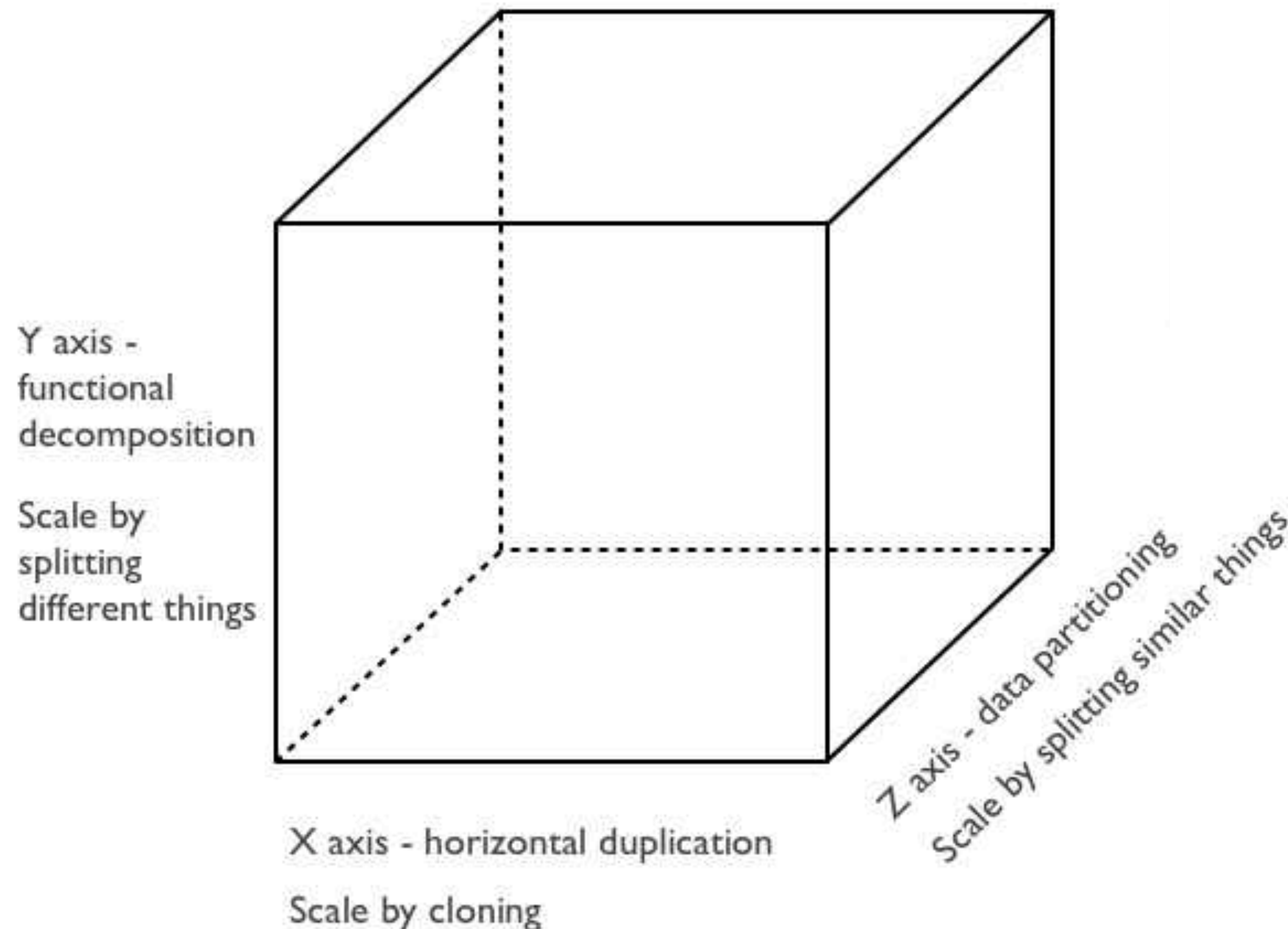
Information

This week (and for all the remaining weeks until the end) there will be no lab assignments

Practical classes will be focused on project support

The scale cube

Revisions from lecture 5



Horizontal duplication

- Duplicate nodes in the cloud

Data partitioning

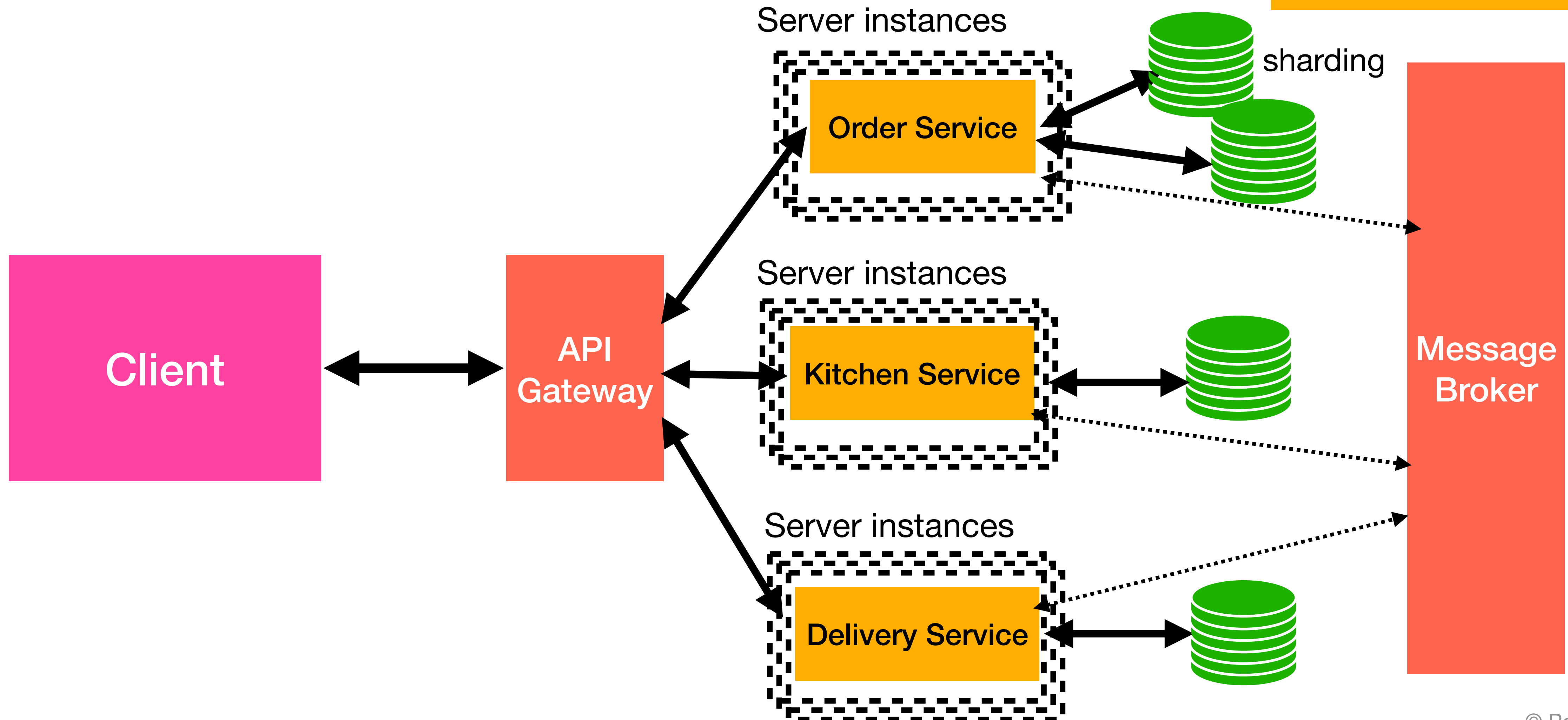
- Customers A to M stored in DB 1
- Customers N to Z stored in DB 2 (data sharding)

Functional decomposition

- Divide the application into services

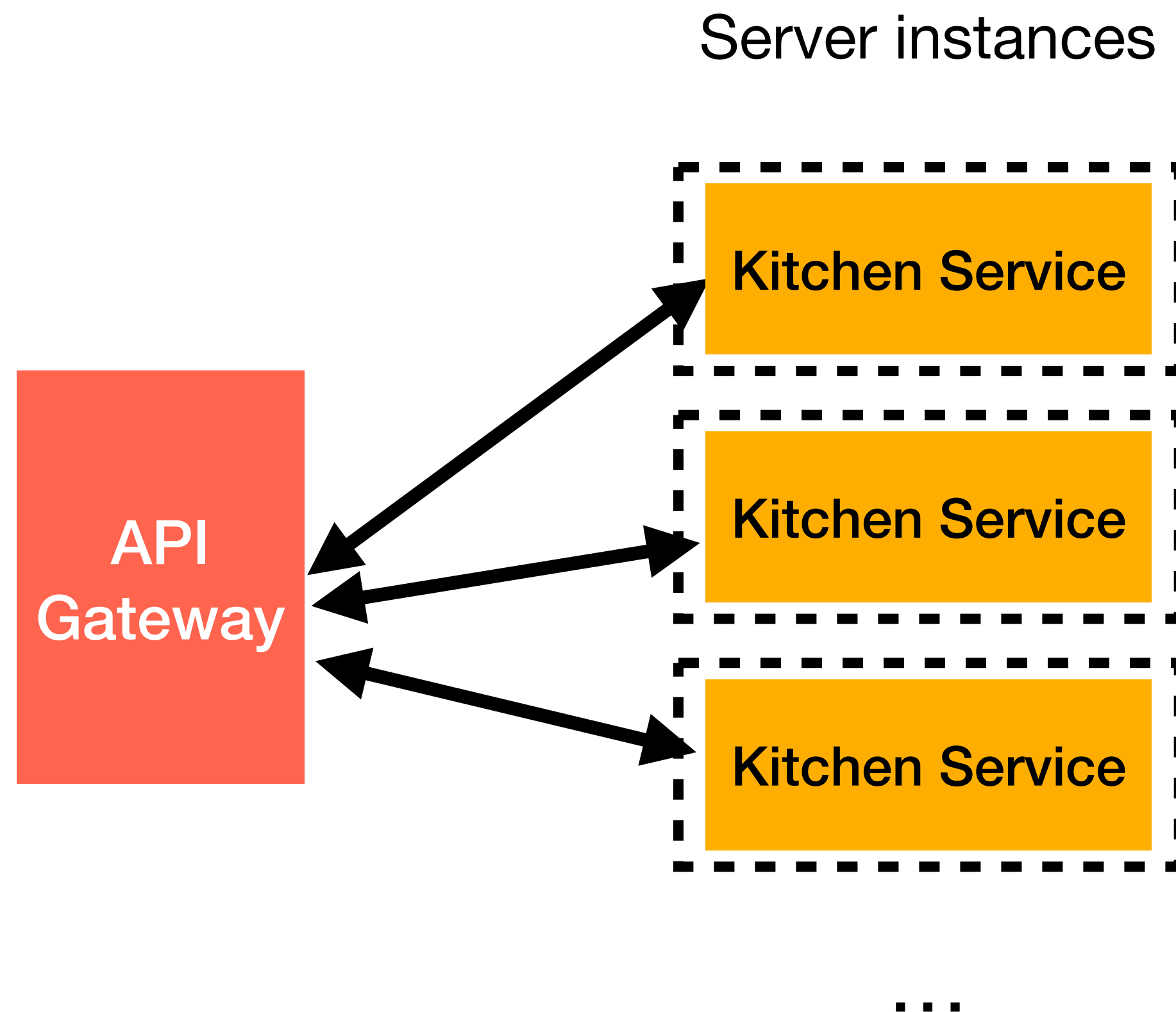
Microservice architecture

Revisions from lecture 5



Microservice architecture

Service Discovery



Multiple instances of the same service deployed in a (elastic) cloud infrastructure:

- The number of services instances and their locations changes dynamically
- Virtual machines and containers are usually assigned a dynamic IP address

Problem:

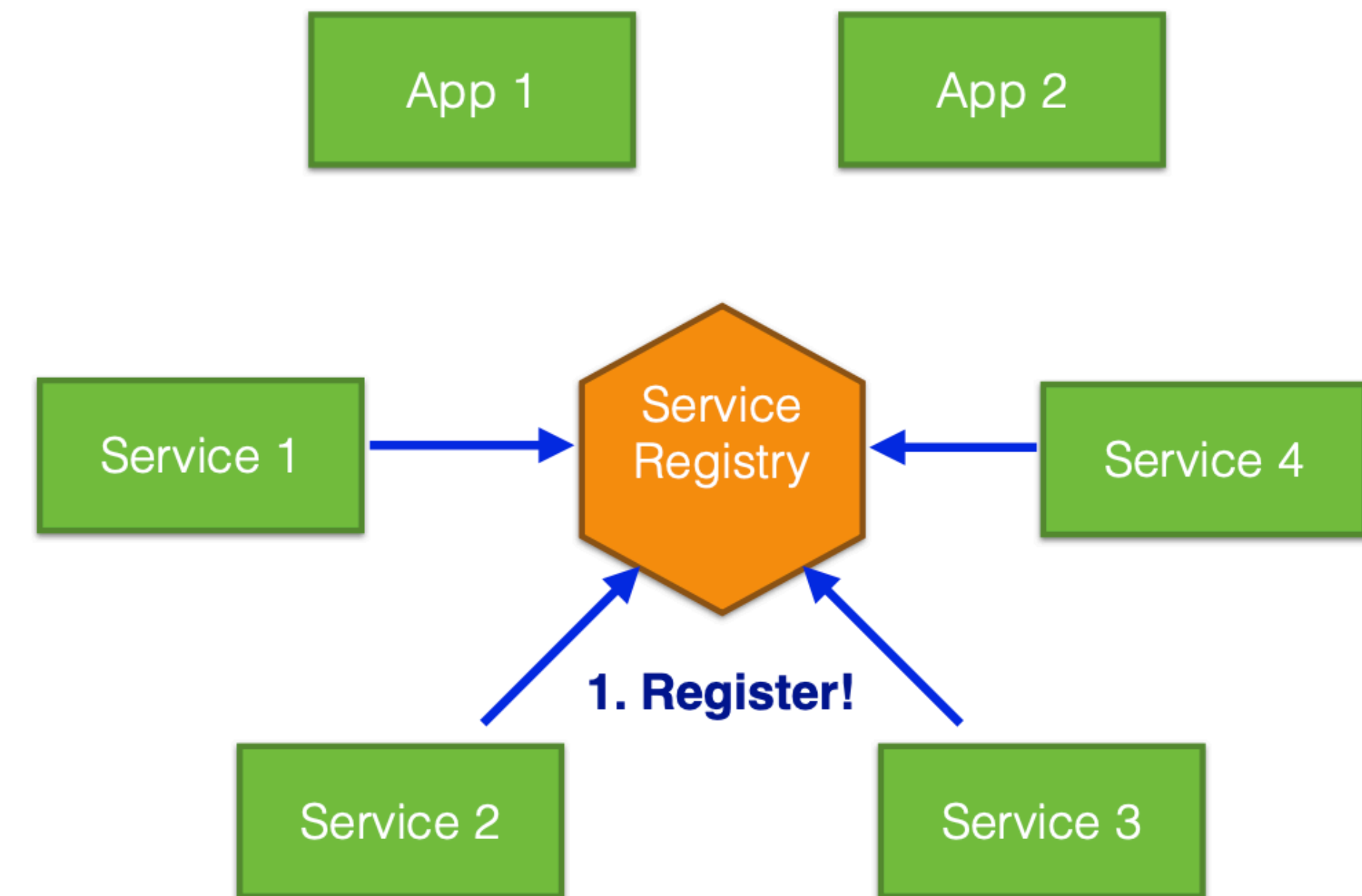
How do clients of the service know about available instances?

Microservice architecture

Service Registry

Clients of a service use either Client-side discovery or Server-side discovery to determine the location of a service instance to which to send requests.

A **service registry** is a database of services, their instances and their locations (optionally, also their health). Must be deployed on fixed and well-known IP addresses



Microservice architecture

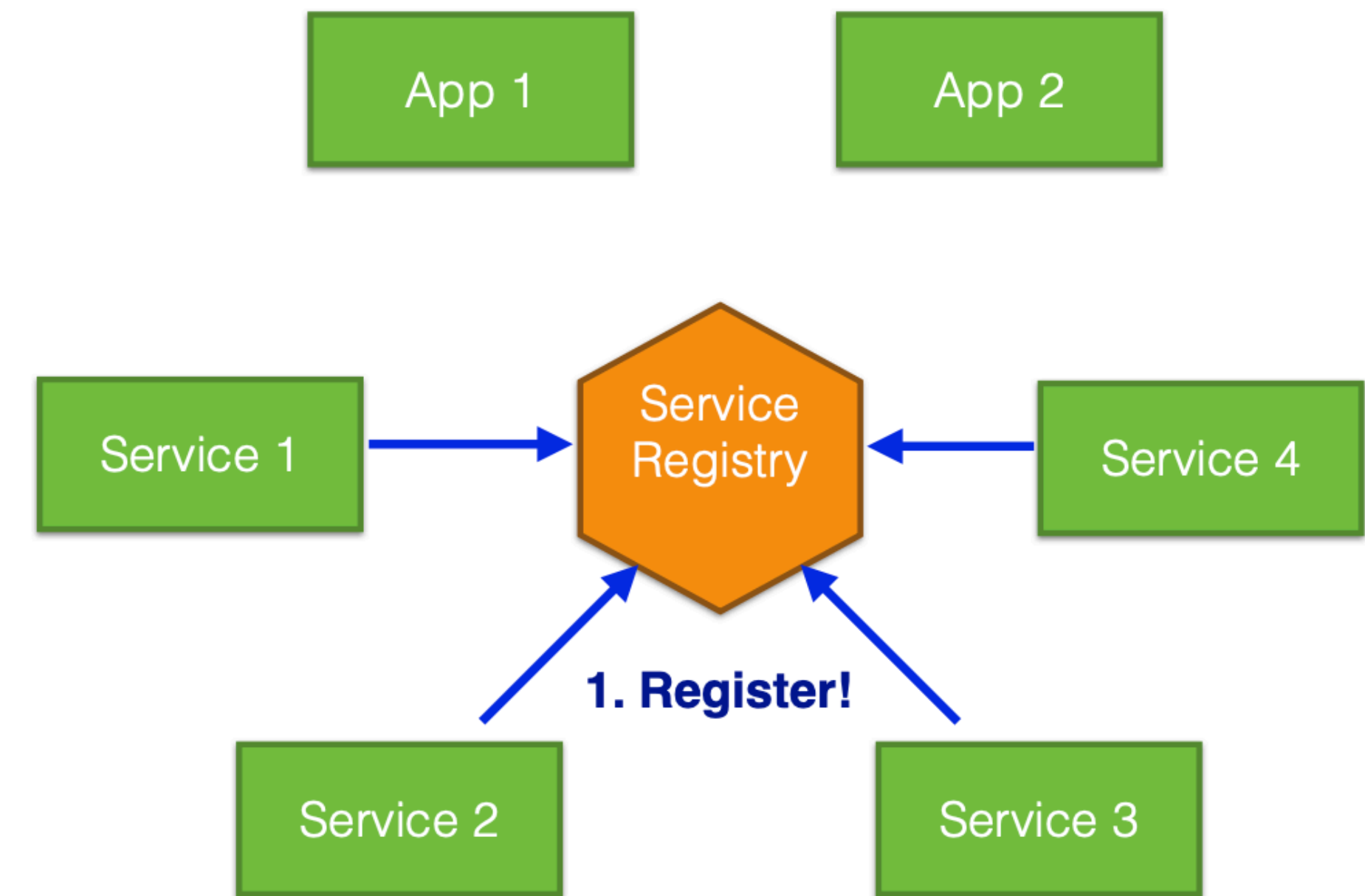
Service Registry

Benefits

- Client of the service and/or routers can discover the location of service instances

Drawbacks:

- Single point of failure
Clients should cache data provided by the service registry but cache data may become stale
- Adds complexity



Microservice architecture

Service Registration

How are server instances registered/unregistered in the Service Registry?

- Register with the service registry on startup and unregister on shutdown
- Service instances that crash must be unregistered
- Service instances that are running but incapable of handling requests must be unregistered

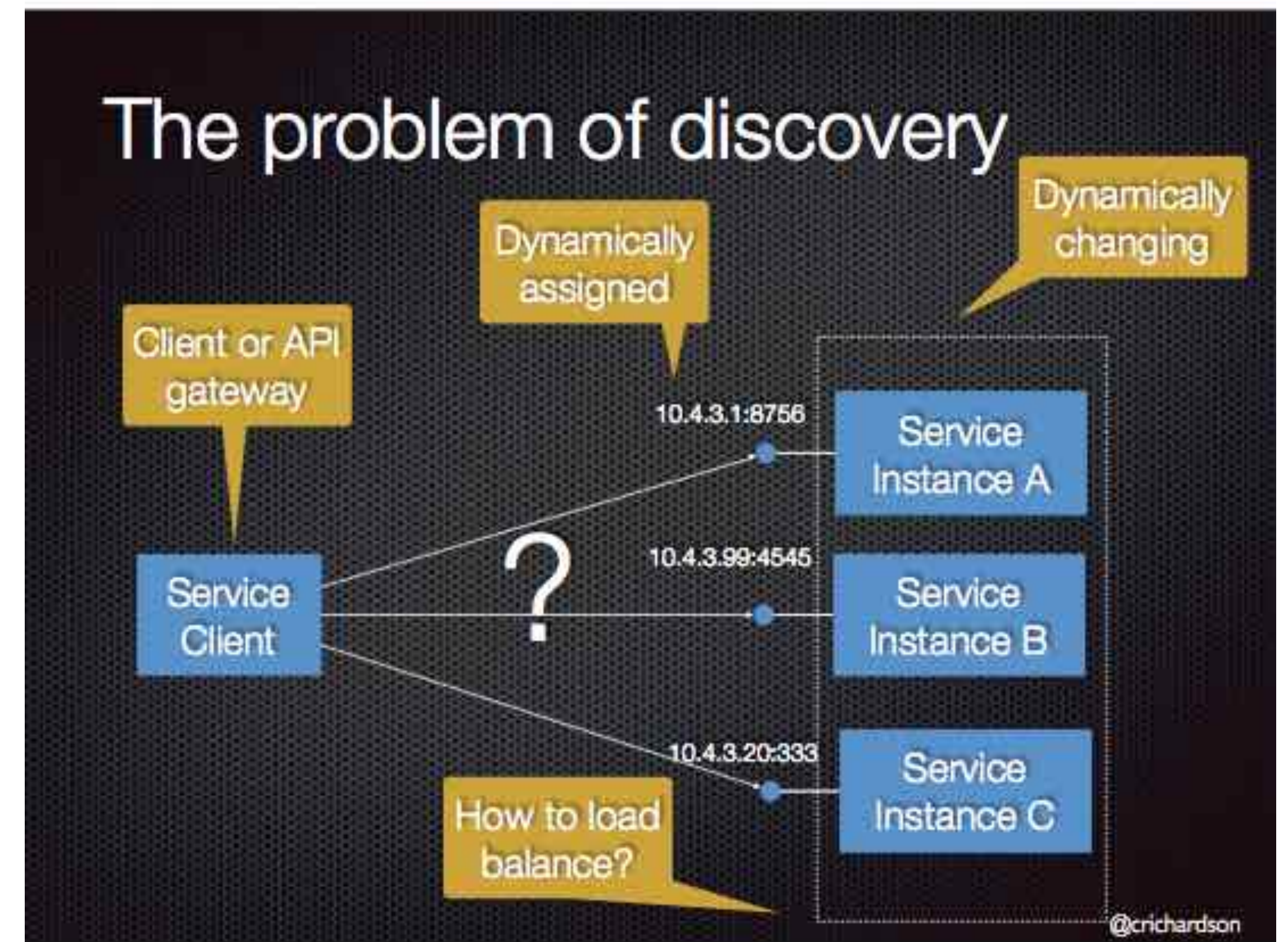
Two options:

- **Self-registration** - instances register themselves
- **3rd party registration service** - a 3rd party registers the service instances with the service registry

Microservice architecture

Service Discovery

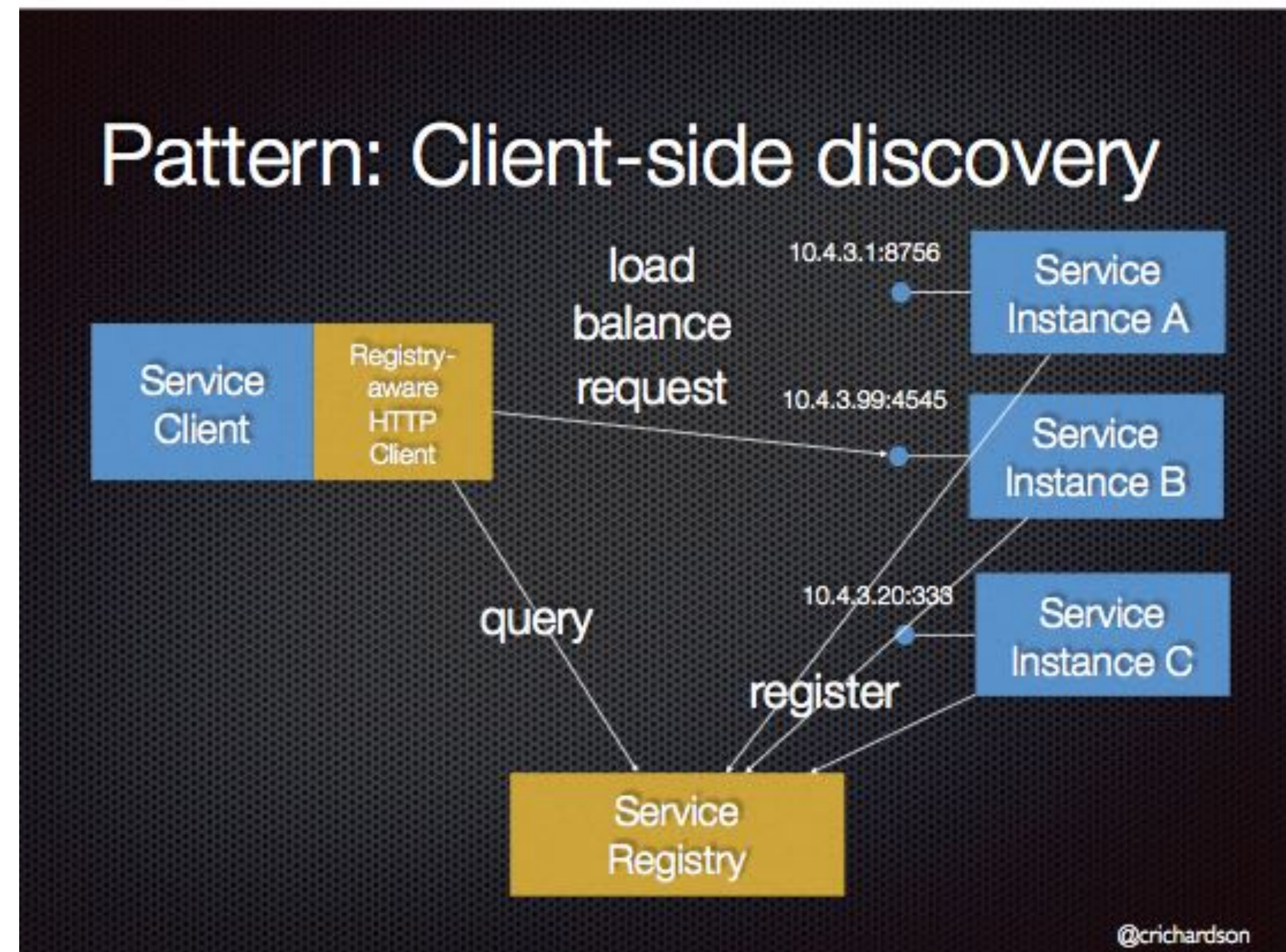
How to discover dynamically assigned, continuously changing server instances?



Microservice architecture

Client-side Service Discovery

The client obtains the location of a service instance by querying a Service Registry



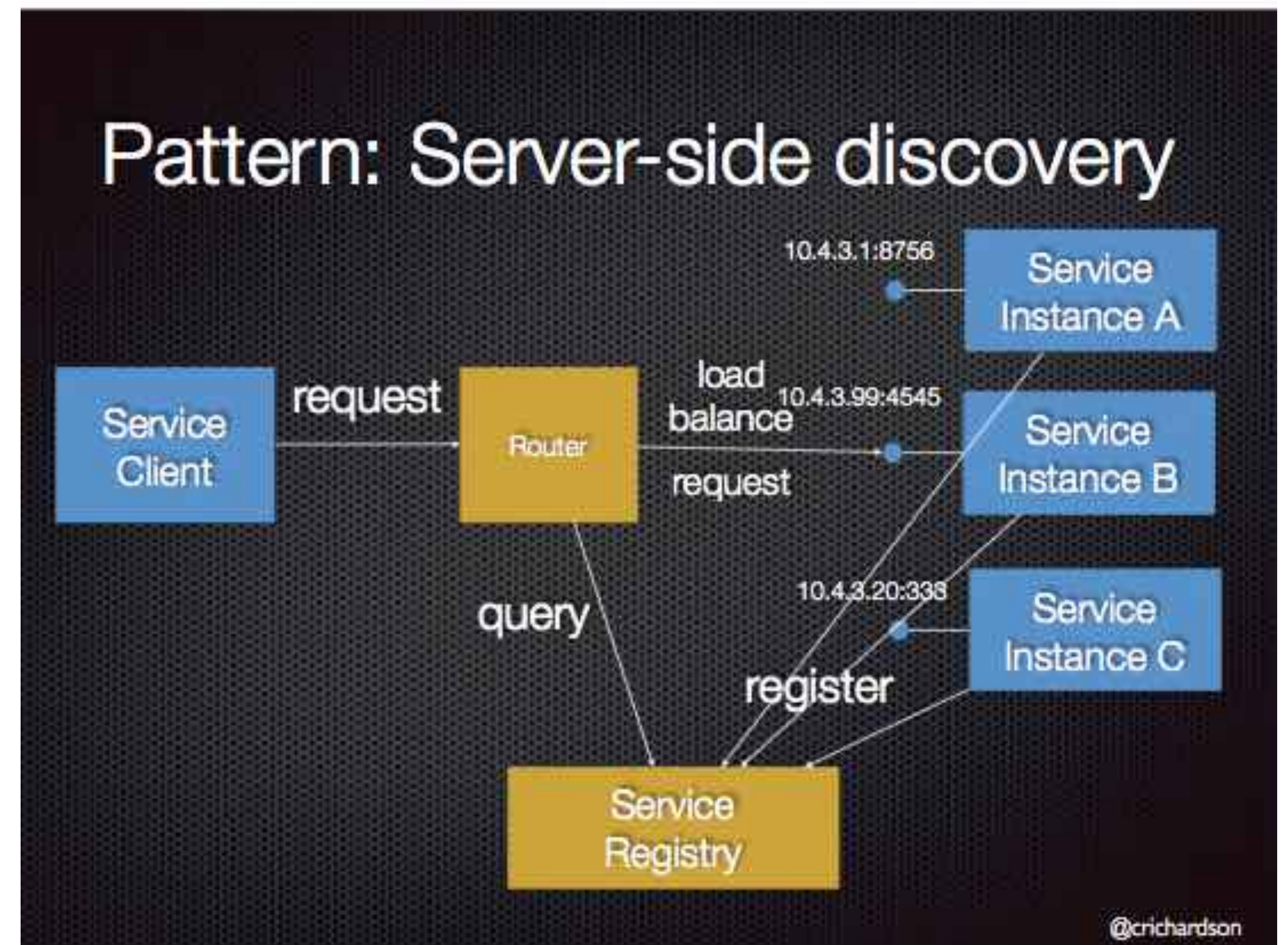
Microservice architecture

Server-side Service Discovery

The client makes a request via a router (*).

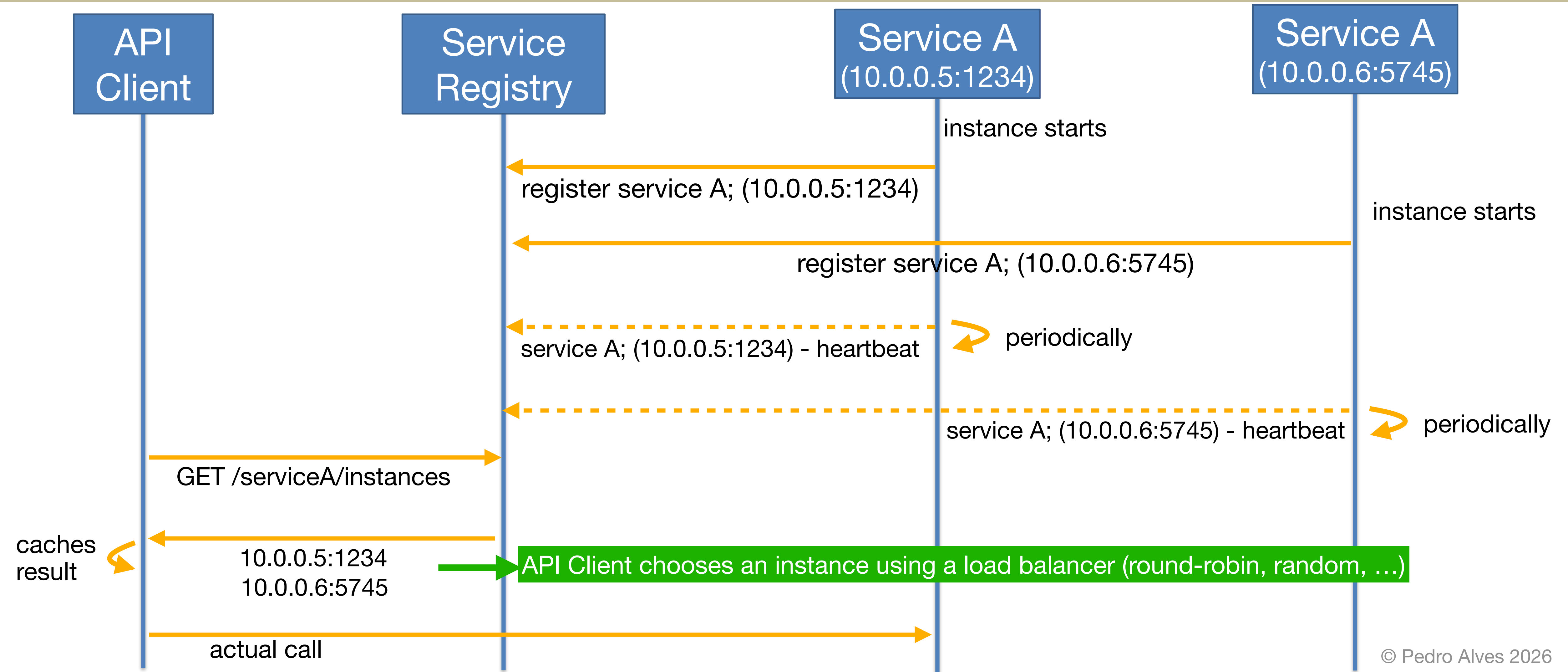
The router obtains the location of a service instance by querying a Service Registry and forwards the request to that location

(*) The router runs at a well-known location



Microservice architecture

Service Registry/Discovery Flow



Microservice architecture

Service Discovery

Exercise

	Client-side discovery	Server-side discovery
Client complexity (low, high)		
System complexity (simple, complex)		
Performance (slower, faster)		
Load balancer (efficient, not so efficient)		
Client coupling to registry (tight, loose)		

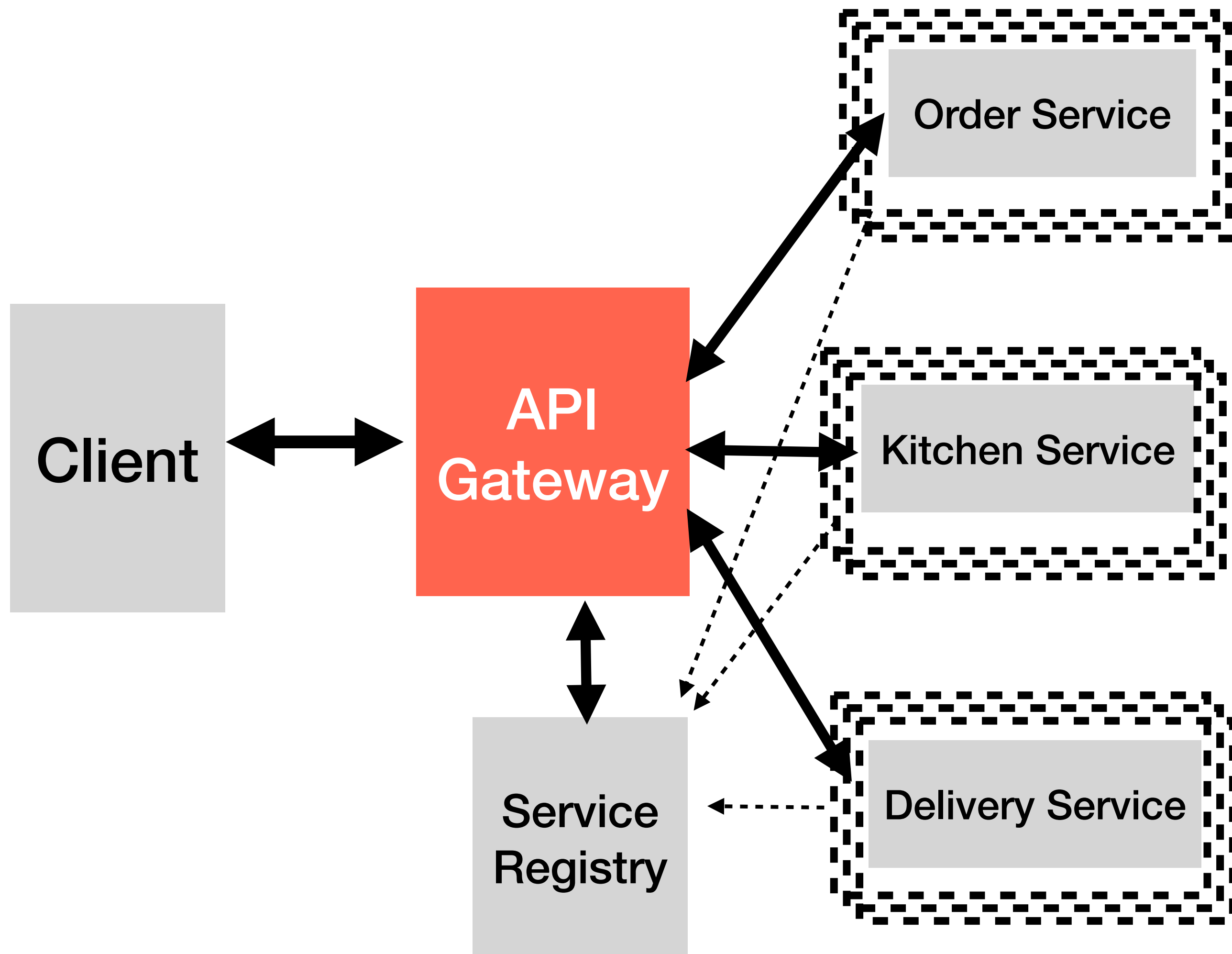
Microservice architecture

Service Discovery

	Client-side discovery	Server-side discovery
Client complexity (low, high)	high	low
System complexity (simple, complex)	simple	complex
Performance (slower, faster)	faster (direct calls)	slower (extra hop)
Load balancer (efficient, not so efficient)	not so efficient	efficient
Client coupling to registry (tight, loose)	tight	loose (client is unaware of the registry service)

Microservice architecture

API Gateway



- Single entry point for all clients
- Routes requests to the appropriate service
- Supports server-side service discovery
- May handle authentication/authorization
- May handle cross-domain queries using the API Composition Pattern

Microservice architecture

API Gateway

Benefits

- Insulates the clients from how the application is partitioned into microservices
- Insulates the clients from the problem of determining the locations of service instances
- Can translate between client-friendly protocols and internal service protocols
- Using the API Composition pattern, reduces the number of requests/roundtrips
- Centralizes cross-cutting concerns in one place: authentication, rate limiting, logging, CORS, ...

Drawbacks

- Potential single point of failure / bottleneck
- For direct requests, adds a network hop
- Risk of becoming a monolith with time

Microservice architecture

Service Registry - Eureka

- Netflix Eureka implements Service Registry and Service Discovery
- OSS Project developed in Java - <https://github.com/Netflix/eureka>
- Supports instances self-registration and both client-side and server-side discovery
- Integrates with Spring through the Spring Cloud project

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2025.0.2</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

ensures all dependencies of
spring cloud are compatible
(eureka, openfeign, etc)

Microservice architecture

Service Registry - Eureka

Eureka Server

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  
</dependency>
```

```
@SpringBootApplication  
@EnableEurekaServer  
class EurekaServerApplication
```



application.properties

```
spring.application.name=eureka-server  
server.port=8761  
eureka.client.register-with-eureka=false  
eureka.client.fetch-registry=false
```


Microservice architecture

Eureka Server HTTP API

Operation	HTTP action	Description
Register new application instance	POST /eureka/v2/apps/appID	Input: JSON/XML payload HTTP Code: 204 on success
De-register application instance	DELETE /eureka/v2/apps/appID/instanceID	HTTP Code: 200 on success
Send application instance heartbeat	PUT /eureka/v2/apps/appID/instanceID	HTTP Code: * 200 on success * 404 if instanceID doesn't exist
Query for all instances	GET /eureka/v2/apps	HTTP Code: 200 on success Output: JSON/XML
Query for all appID instances	GET /eureka/v2/apps/appID	HTTP Code: 200 on success Output: JSON/XML
Query for a specific appID/instanceID	GET /eureka/v2/apps/appID/instanceID	HTTP Code: 200 on success Output: JSON/XML
Query for a specific instanceID	GET /eureka/v2/instances/instanceID	HTTP Code: 200 on success Output: JSON/XML

Microservice architecture

Eureka Server UI



HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2024-10-06T17:59:13 +0100
Data center	default	Uptime	01:11
		Lease expiration enabled	true
		Renews threshold	8
		Renews (last min)	16

DS Replicas

[localhost](#)

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDERSERVICE	n/a (1)	(1)	UP (1) - joaos-air.lan:OrderService:0
PRODUCTSERVICE	n/a (1)	(1)	UP (1) - joaos-air.lan:ProductService:0
SOCIALAPP	n/a (1)	(1)	UP (1) - joaos-air.lan:SocialApp:8080
USERSERVICE	n/a (1)	(1)	UP (1) - joaos-air.lan:UserService:0

Microservice architecture

Service Registration - Eureka

Eureka Client (instance self-registration)

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  
</dependency>
```

application.properties

```
spring.application.name=my-service  
  
server.port=0  
  
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Run on a random available port. This is key for elastic deployments

Service Registration - Eureka

Running the service standalone during development (useful for debugging)

application-local.properties

```
eureka.client.enabled=false  
server.port=8081
```

← Don't connect to eureka server

← Run on a fixed port

Run the service with “local” Spring profile

```
mvn spring-boot:run -Dspring-boot.run.profiles=local
```

Active profiles:

local

← Run configuration in IntelliJ

Microservice architecture

Service Discovery - Eureka

Add the eureka-client dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Annotate the RestClient bean with @LoadBalanced

```
@Bean
@LoadBalanced 
fun restClient(builder: RestClient.Builder): RestClient {
    return builder.build()
}
```

```
@Service
class OrderService(private val restClient: RestClient) {

    fun callOrderService() = restClient.get()
        .uri("http://ORDER-SERVICE/orders")
        .retrieve().body(String::class.java)
}
```

Eureka client will:

- lookup ORDER-SERVICE instances in the service registry
- choose one using the load-balancer
- point restClient to that instance 

Microservice architecture

Service Discovery - Eureka

Instead of RestClient or HttpExchange, you can (should) use OpenFeign which integrates directly with the Eureka Client

```
@FeignClient(name = "order-service")
interface OrderServiceFeignClient {
    @GetMapping("/orders")
    fun getOrders(): OrdersResponse
}
```



Automatically discovers an instance of the order-service and uses it

Needs this dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Microservice architecture

Service Discovery - Eureka

If you need more control (get the instance address yourself), you can use the LoadBalancerClient

```
@Service
class OrderServiceCaller(
    private val loadBalancerClient: LoadBalancerClient,
    private val restClient: RestClient
) {

    fun callOrderService(): String {
        // 1. Choose an instance via Eureka + LoadBalancer
        val instance = loadBalancerClient.choose("order-service")
            ?: throw IllegalStateException("No instances available for order-service")

        // 2. Build the real URL
        val url = "${instance.uri}/orders"

        // 3. Call the service
        return restClient.get()
            .uri(url)
            .retrieve()
            .body(String::class.java)
    }
}
```


API Gateway - Spring Cloud Gateway

Spring Cloud provides an API Gateway implementation - Spring Cloud Gateway

- Provides routing, filtering, and load balancing for microservices
- Integrates with service discovery (e.g., Eureka)

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-gateway-mvc</artifactId>  
</dependency>
```


Microservice architecture

Spring Cloud Gateway

Key Features

- **Dynamic routing** based on path, headers, or predicates
- **Built-in filters** for:
 - authentication & authorization
 - rate limiting
 - logging & tracing
- **Load balancing** via service discovery
- **Circuit breaking and resilience**

Microservice architecture

Spring Cloud Gateway

Implementing routing rules programmatically

@Configuration

class GatewayConfig {

@Bean

fun kitchenServiceRoute(): RouterFunction<ServerResponse> =

GatewayRouterFunctions.route("kitchen-service")

.route(RequestPredicates.path("/kitchen/**"),

HandlerFunctions.http("lb://kitchen-service"))

.filter(BeforeFilterFunctions.stripPrefix(1))

.build()

}

}

Create a named route for monitoring purposes

Route all /kitchen/** requests to the kitchen-service instance (resolved via Eureka Registry)

before forwarding, strips the first path segment. /kitchen/orders/42 becomes /orders/42

Microservice architecture

Spring Cloud Gateway

Implementing routing rules declaratively

application.yml

```
spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: kitchen-service
              uri: lb://kitchen-service
              predicates:
                - Path=/kitchen/**
              filters:
                - StripPrefix=1
```


Microservice architecture

Spring Cloud Gateway

If you use the lb: prefix (load balancer), it will connect to an Eureka server - must include it in the configuration

application.yml

```
eureka:  
  client:  
    service-url:  
      defaultZone: http://localhost:8761/eureka/
```


Microservice architecture

OpenAPI

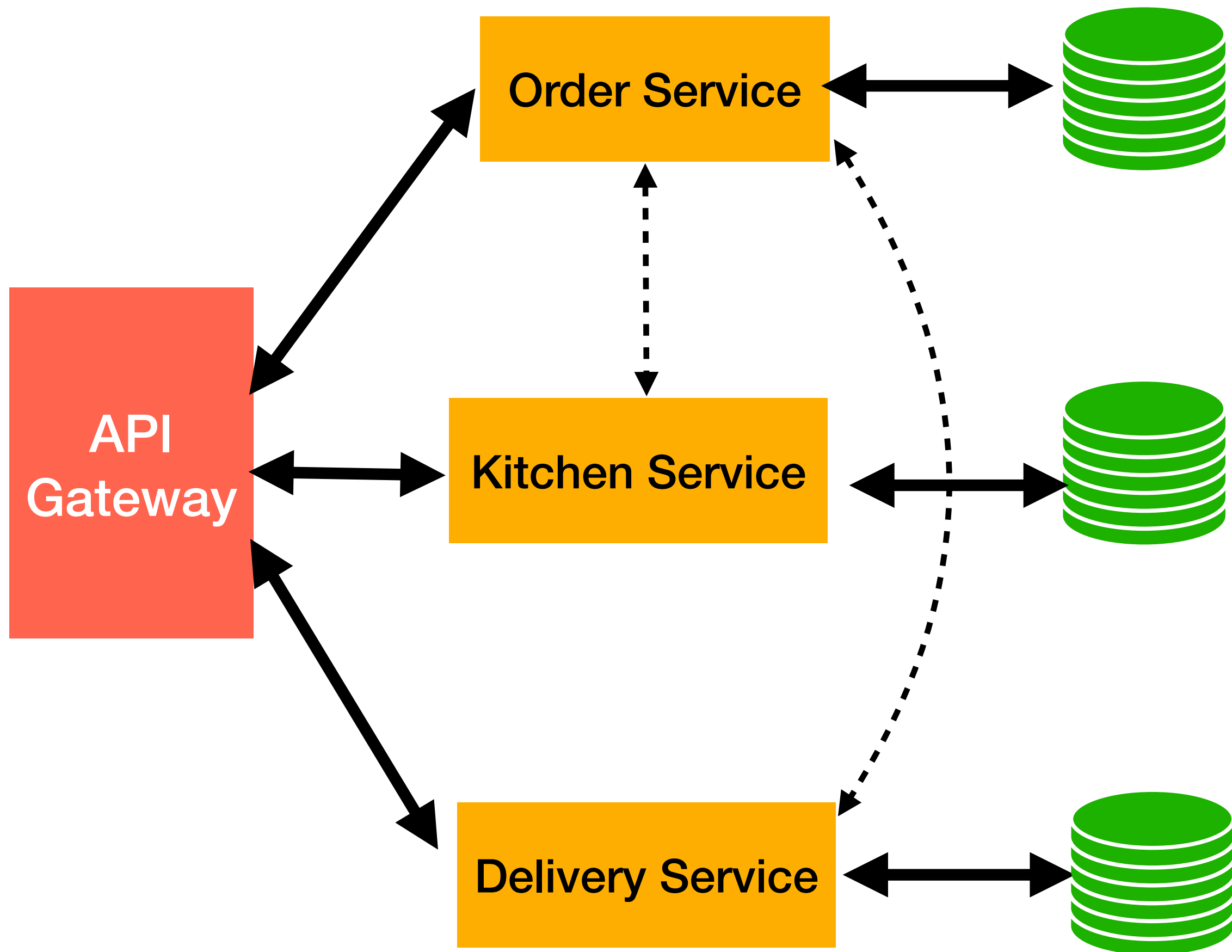
OpenAPI documentation should be generated for all the services and aggregated through the API Gateway

1. Document all your services following the approach described in lecture 2 (annotating the API interface)
2. On the gateway, configure springdoc.swagger-ui.urls pointing to each service's /v3/api-docs (proxied through the gateway routes)
3. The Swagger UI on the gateway becomes the single entry point with a service dropdown

(api-gateway) application.properties

```
springdoc:  
  swagger-ui:  
    path: /swagger-ui.html  
    urls:  
      - name: order-service  
        url: /orders/v3/api-docs  
      - name: kitchen-service  
        url: /kitchen/v3/api-docs  
      - name: delivery-service  
        url: /deliveries/v3/api-docs
```

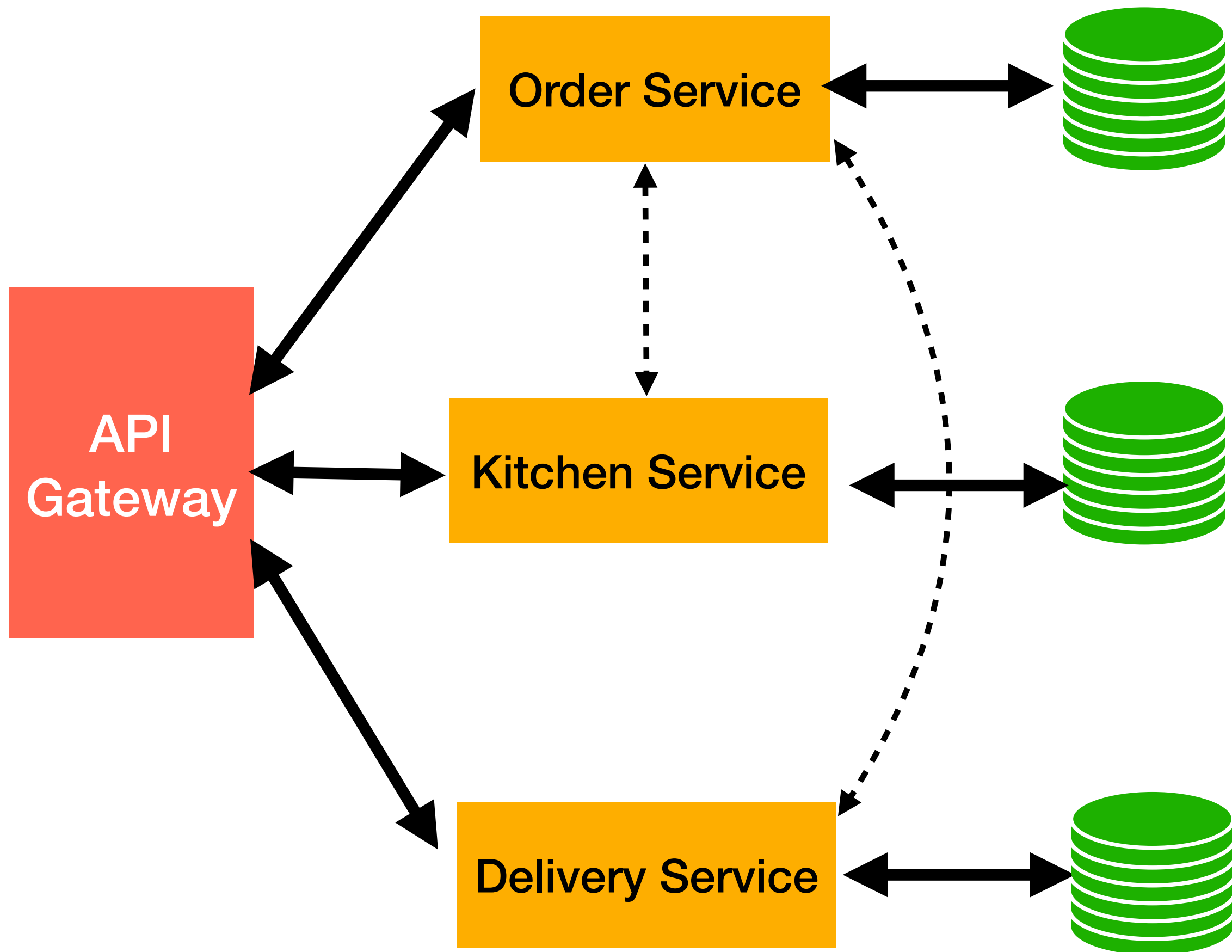
Inter-service communication



Example 1: Place an order

1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. The **Order Service** creates the order
4. The **Order Service** asks the **Kitchen Service** to prepare it
5. Later, the **Order Service** asks the **Delivery Service** to deliver it

Inter-service communication



Example 2: Order Status

1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. **Order Service** may need:
 - kitchen status from **Kitchen Service**
 - delivery status from **Delivery Service**
4. **Order Service** returns an aggregated response

Inter-service communication - synchronous

Approach A: Direct **synchronous** communication

One service directly calls another, usually over HTTP

1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. The **Order Service** creates the order
4. The **Order Service** asks the **Kitchen Service** to prepare it
5. Later, the **Order Service** asks the **Delivery Service** to deliver it

Order Service calls Kitchen Service and **waits** for the response

Order Service calls Delivery Service and **waits** for the response

Inter-service communication - synchronous

In Spring, the easiest way to implement synchronous communication is using a FeignClient

```
@FeignClient(name = "order-service")
interface OrderServiceFeignClient {
    @GetMapping("/orders")
    fun getOrders(): OrdersResponse
}
```



Automatically discovers an instance of the order-service and uses it

Note: For inter-service communication, no need for going through the API Gateway

Inter-service communication - asynchronous

Approach B: Asynchronous communication using a message broker

A service publishes a message or event, and another service consumes it

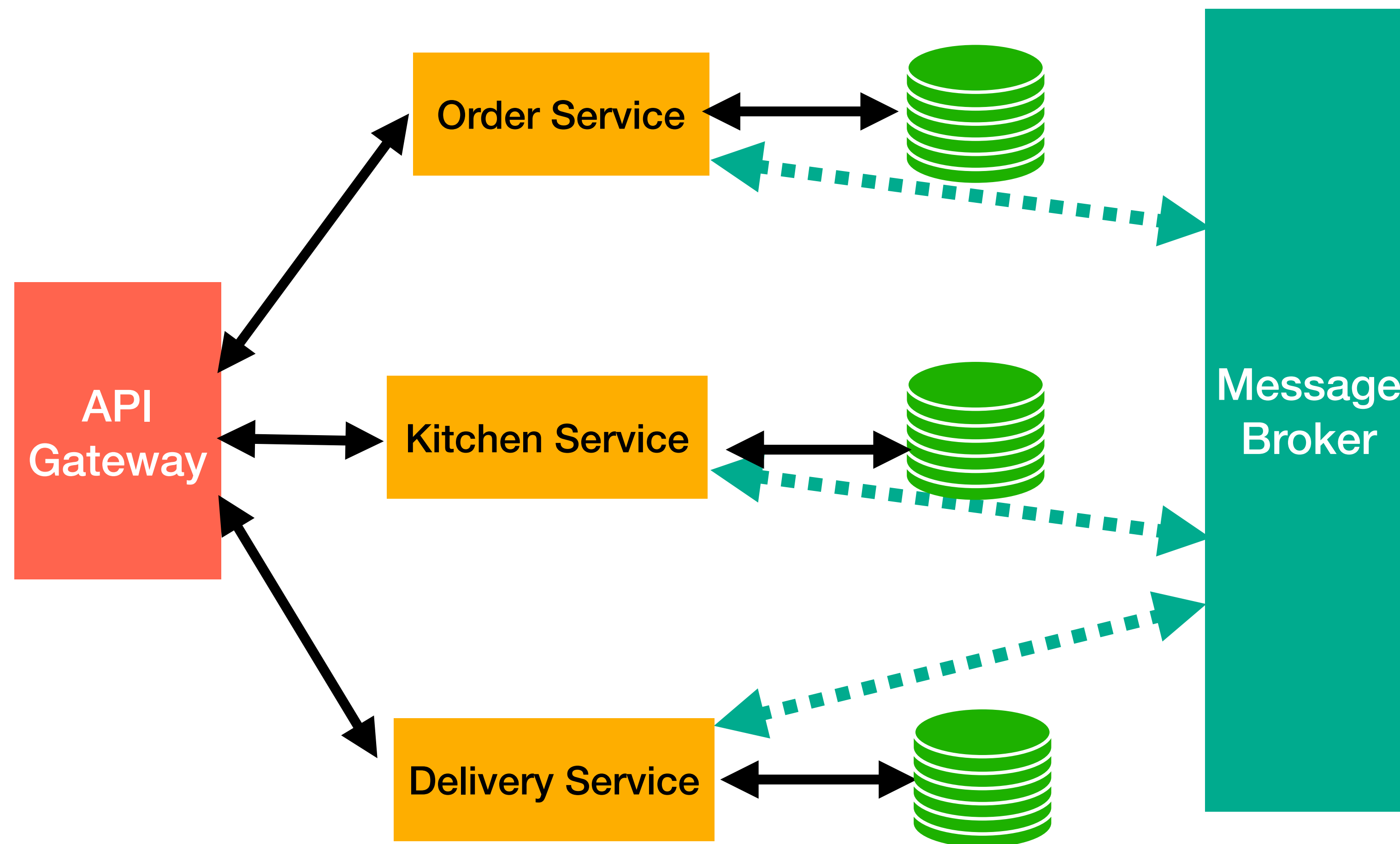
1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. The **Order Service** creates the order
4. ~~The **Order Service** asks the **Kitchen Service** to prepare it~~
5. ~~Later, the **Order Service** asks the **Delivery Service** to deliver it~~

Order Service publishes `OrderCreated`

- Kitchen Service consumes it and starts preparation
- Kitchen Service publishes `OrderReady`
- Delivery Service consumes it and starts delivery

Inter-service communication - asynchronous

Approach B: Asynchronous communication using a message broker



Inter-service communication

Example 1: Place an order

1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. The **Order Service** creates the order
4. The **Order Service** asks the **Kitchen Service** to prepare it
5. Later, the **Order Service** asks the **Delivery Service** to deliver it

No real-time dependency:
asynchronous communication

Example 2: Order Status

1. The client calls the **API Gateway**
2. The gateway forwards the request to the **Order Service**
3. **Order Service** may need:
 - kitchen status from **Kitchen Service**
 - delivery status from **Delivery Service**
4. **Order Service** returns an aggregated response

Real-time dependency:
synchronous communication

Microservice architecture

Inter-service communication

Exercise

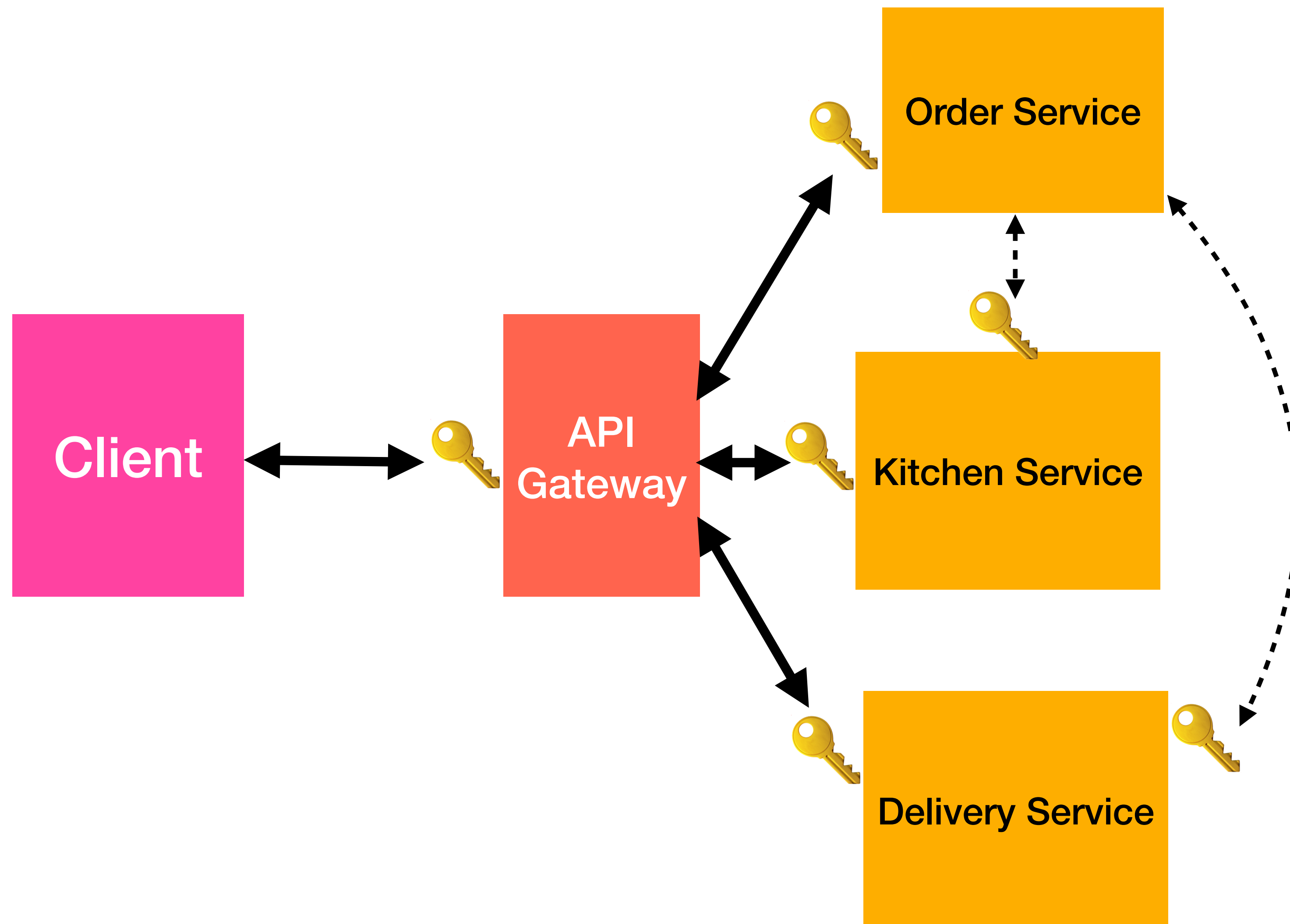
	Synchronous communication	Asynchronous communication
Interaction style (Request/response;Publish/subscribe)		
Coupling between services (loose;tight)		
Response time (immediate;delayed)		
Consistency model (strong,relaxed)		
Complexity (simple,complex)		

Microservice architecture

Inter-service communication

	Synchronous communication	Asynchronous communication
Interaction style (Request/response;Publish/subscribe)	Request/response	Publish/subscribe
Coupling between services (loose;tight)	Tight	Loose
Response time (immediate;delayed)	Immediate (blocking)	Delayed (non-blocking)
Consistency model (strong,relaxed)	Stronger consistency	Relaxed (eventual) consistency
Complexity (simple,complex)	Simpler	More complex

Microservice architecture Authentication



Users must be authenticated to enter the front gate (API Gateway)

- How do they authenticate?
- How is their identity propagated across the system?

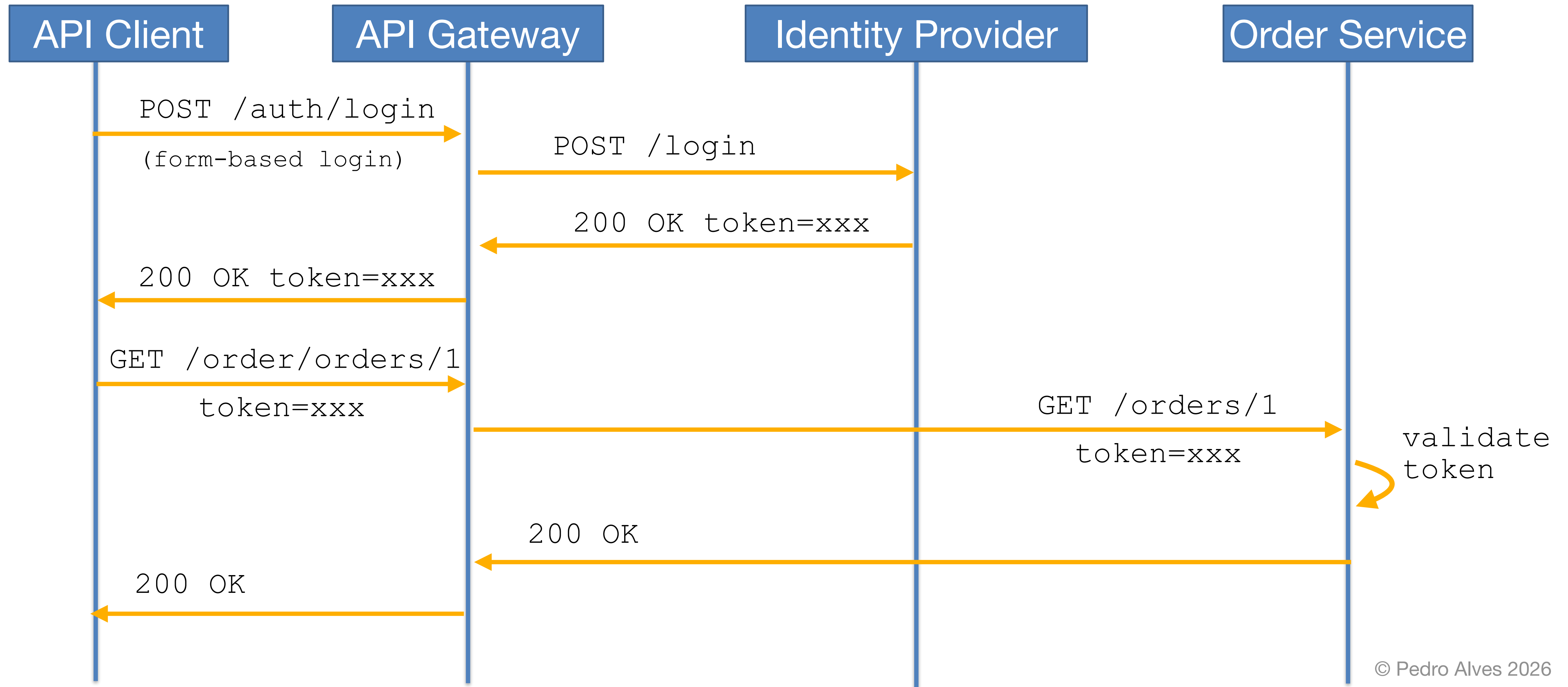
Microservice architecture

Authentication

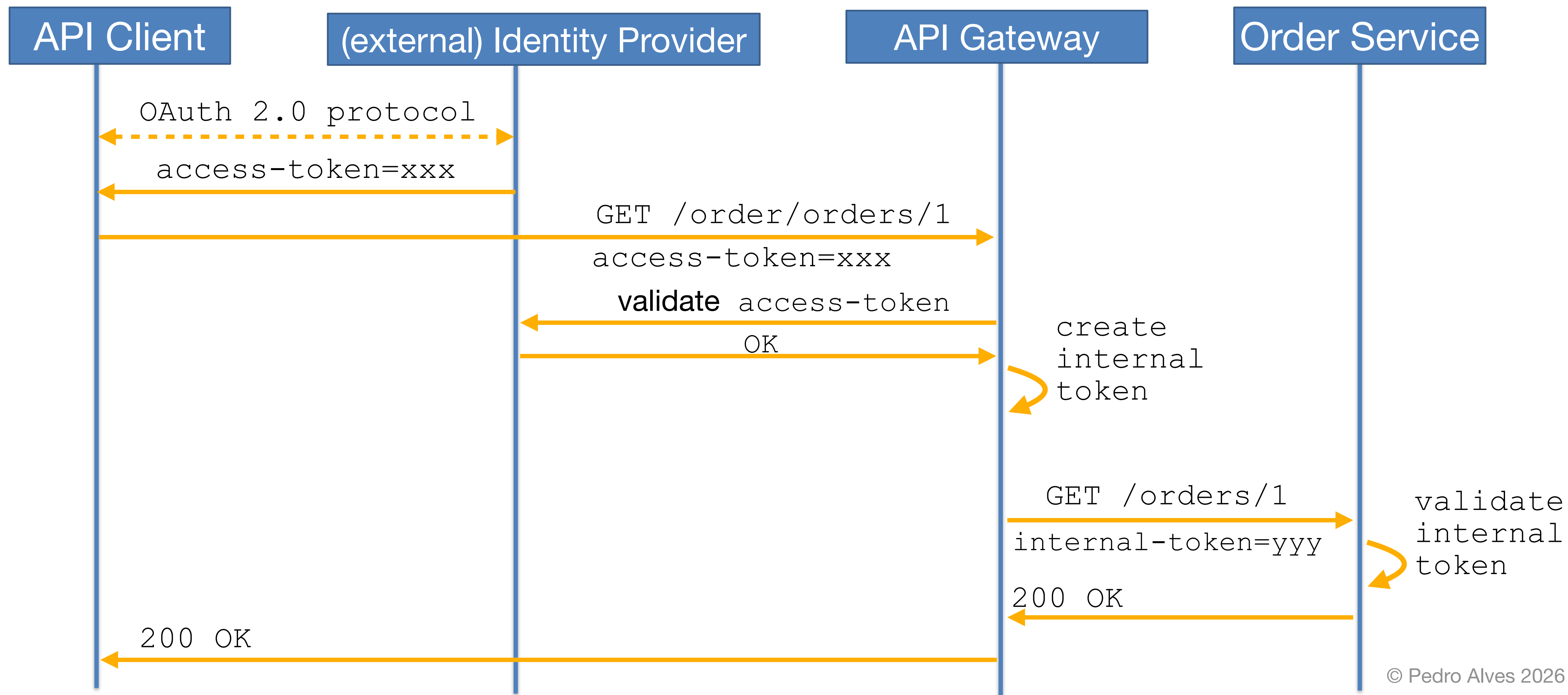
- The client application connects to an IdP (Identity Provider) which can be internal or external
- The IdP validates credentials and returns a token
- Calls API Gateway (or backend) with the token
- Gateway / service validates token
- Request forwarded to microservices (including token)

Note: If the IdP is external, the external token may be converted to an internal token

Example authentication flow with internal IdP



Example authentication flow with external IdP



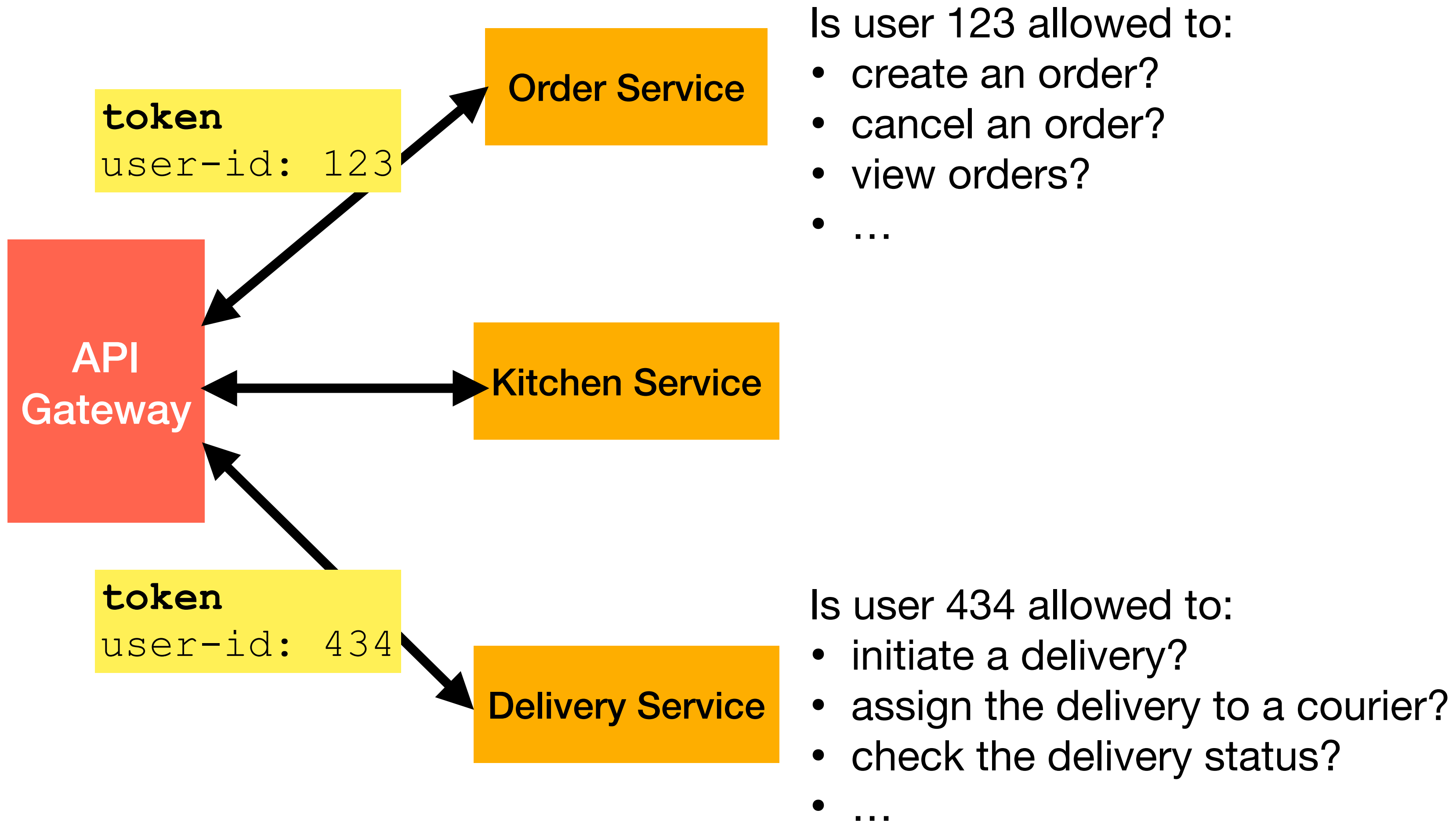
Microservice architecture

Authentication

- The tokens may be:
 - **Self-contained** - may be validated “offline”, they contain user identity information and are digitally signed (using public/private keys or symmetric shared key), usually implemented with JWT
 - **Opaque** - do not contain user information, and are instead represented by a random, unique identifier that is mapped to user information in a auth server (equivalent to the JSESSIONID). Upon receiving the token, it must be validated “online”, calling an endpoint in the auth server (token introspection)
- The external-to-internal token conversion may be done by a specific service, not the API Gateway
- The API Gateway may pass only the “*user id*” and not the whole token to the microservices (requires secure inter-service communication)

Microservice architecture

Authorization



Authorization approaches

- **Decentralized authorization (per service)** - Each service maps the user ID / claims to authorization rules
 - Benefits: Full service autonomy, no external dependency
 - Drawbacks: Duplicates authorization logic and data, hard to keep policies consistent across services
- **Self-contained token (e.g. JWT with claims)** - Token includes authorization data (roles, scopes, etc.)
 - Benefits: Scalable, fast, local decision-making
 - Drawbacks: Vulnerable to attacks, hard to revoke permissions
- **Centralized authorization service** - Services call a central service to check permissions
 - Benefits: Strong consistency of policies, possible to revoke permissions
 - Drawbacks: Requires additional network calls, potential bottleneck

Access control models

- Role-based Access Control
- Model-based Access Control
- Capability-based Access Control

Role-based access control

In microservice architectures, user roles are usually included in the tokens and propagated across the system (self-contained token)

```
{
  "header": {
    ...
  },
  "payload": {
    "sub": "1234567890",
    "name": "John Doe",
    "roles": [ "staff", "manager" ]
  },
  "signature": "SflKxwRJSMeKKF2QT4fw..."
}
```

There is no standard “official” way to include user roles in JWT

Most commonly used - `roles` attribute

Model-based access control

revision from lecture 5

MBAC rules are dynamic and depend on multiple factors

- User attributes (id, department, ownership)

Example: Only users in the Finance department can view payroll data

- Resource attributes (owner, status)

Example:

These can't be stored in tokens

- Context (time, location, ...)

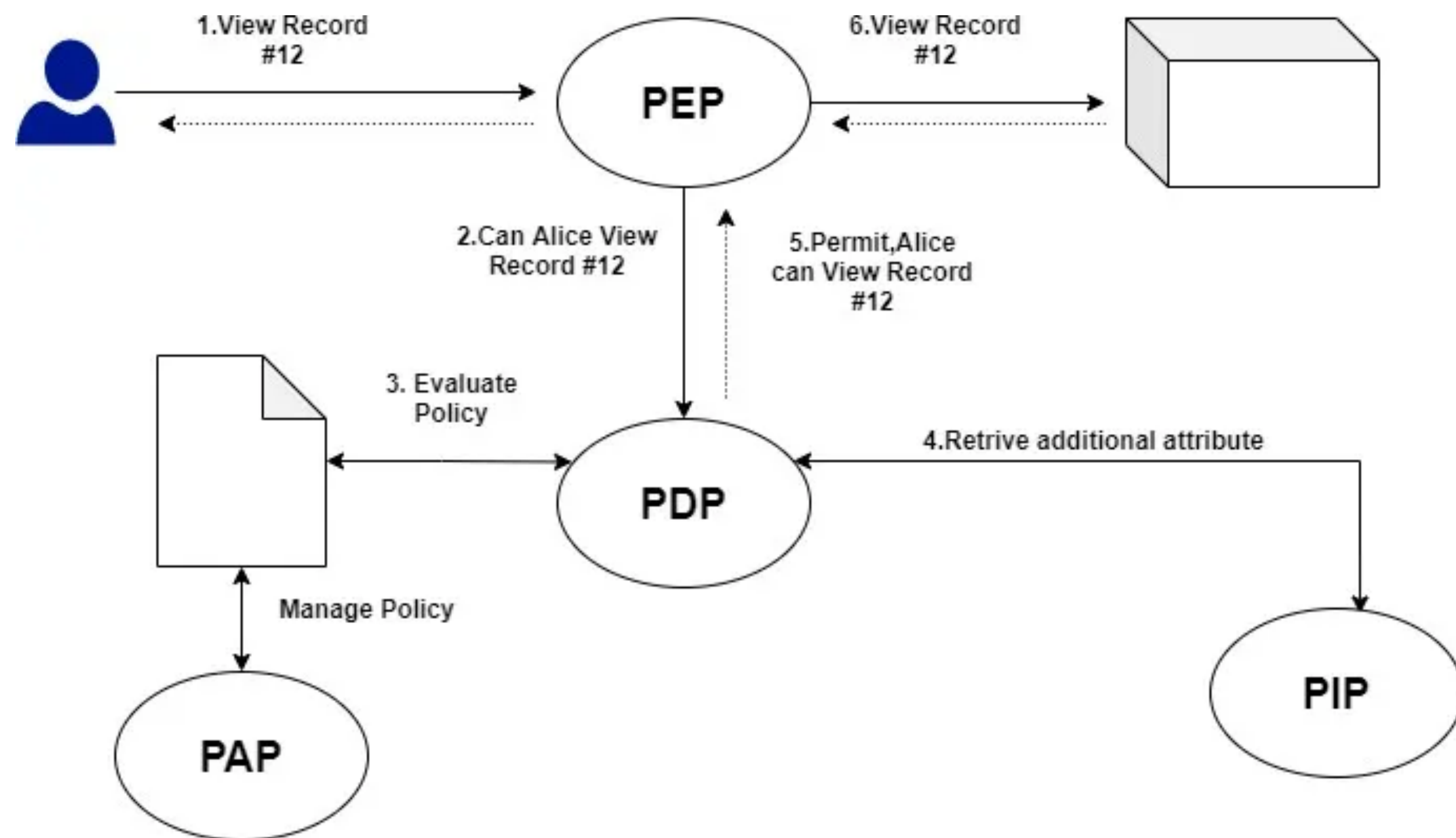
Example: Sensitive data accessible only from office IP addresses

- Relationships (user $\leftrightarrow$ resource, user $\leftrightarrow$ team)

Example: Managers can view reports submitted by their team members

Model-based access control

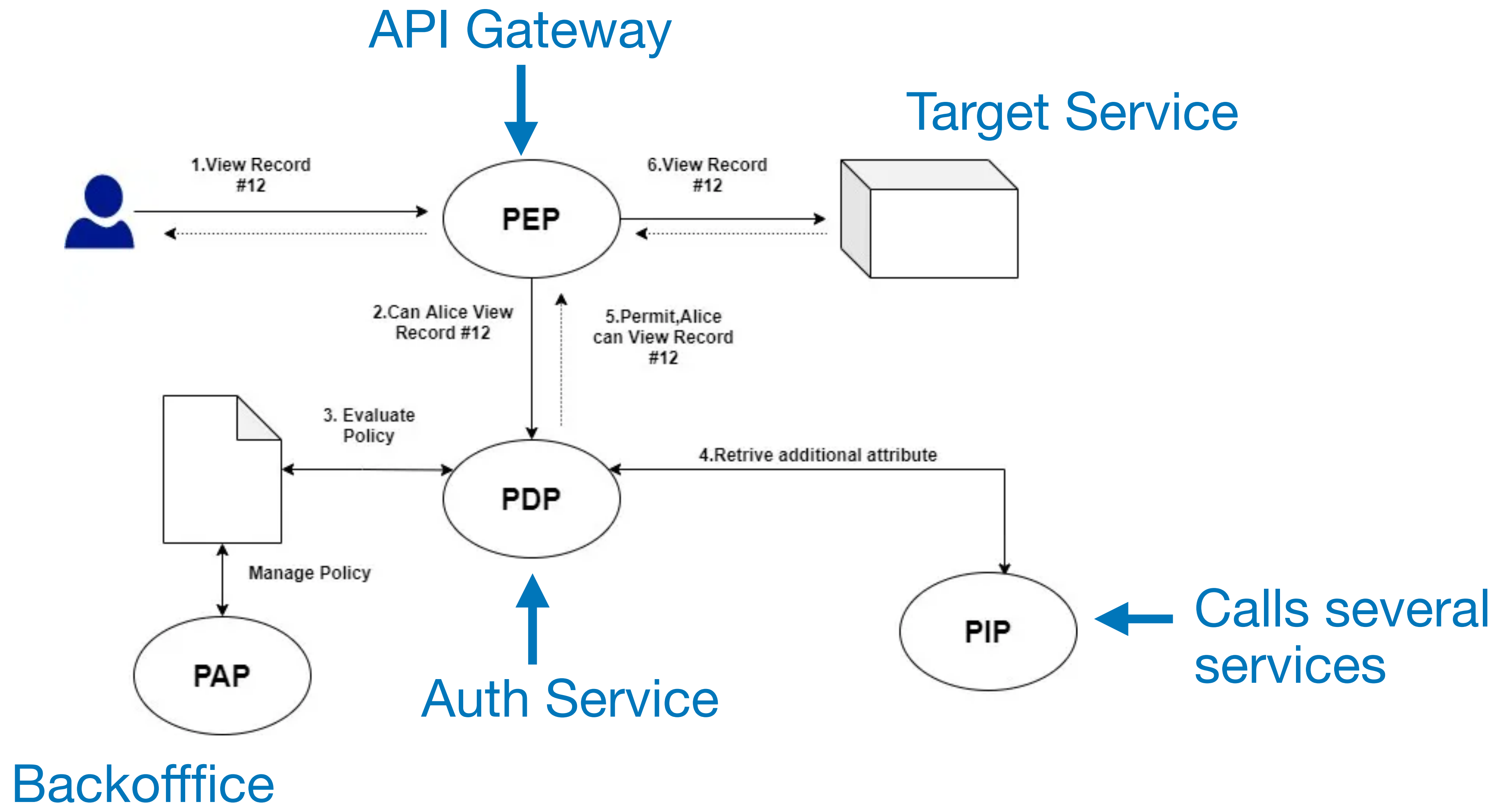
XACML Architecture



- **PEP** (Policy Enforcement Point) intercepts requests
- **PDP** (Policy Decision Point) evaluates policies against attributes
- **PIP** (Policy Information Point) provides attributes needed for decisions
- **PAP** (Policy Administration Point) manages access authorization policies through a backoffice/config repo

Microservice architecture

Model-based access control



Model-based access control

To avoid multiple network calls and increased complexity, MBAC can be locally enforced on each microservice, using the same techniques explained in lesson 5 (@PreAuthorize, Capabilities encoded in annotations, ...)

```
@PostMapping("/posts/{id}")
@AllowedForReplaceBook
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost):
    ResponseEntity<BlogPost> {
    ...
}
```

← checks if user is admin or book owner

Drawback: hard to keep policies consistent across services

Capability-based access control

- A capability refers to an object and a set of rights
 - Users that own it can perform the operation described in the capability
 - Example: a pair ("/etc/pass", O_RDWR) gives readwrite access
- In JWT, capabilities are usually stored in the scope attribute (borrowed from OAuth2)

```
{  
  "header": { ... },  
  "payload": {  
    "sub": "1234567890",  
    "name": "John Doe",  
    "scope": "read:orders write:orders"  
  },  
  "signature": "SflKxwRJSMeKKF2QT4fw..."  
}
```



Capability-based access control

```
{  
  "header": { ... },  
  "payload": {  
    "sub": "1234567890",  
    "name": "John Doe",  
    "scope": "read:orders write:orders"  
  },  
  "signature": "SflKxwRJSMeKKF2QT4fw..."  
}
```



```
@Service  
class OrderService(private val orderRepository: OrderRepository) {
```



```
  @CanReadOrders  
  fun getOrders() = orderRepository.findAll()
```



```
  @CanWriteOrders  
  fun createOrder(order: OrderDTO) = ...  
}
```

Capability-based access control

```
@Service
class OrderService(private val orderRepository: OrderRepository) {

    @CanReadOrders
    fun getOrders() = orderRepository.findAll()
}
```

```
@Target(AnnotationTarget.FUNCTION, AnnotationTarget.CLASS)
@Retention(AnnotationRetention.RUNTIME)
@PreAuthorize("@capabilitiesService.canReadOrders(authentication)")
annotation class CanReadOrders
```

```
@Service
class CapabilitiesService {

    fun canReadOrders(principal: Principal): Boolean =
        "read:orders" in (principal as UserAuthToken).capabilities
}
```

Note: Requires a Filter that reads the JWT capabilities and stores them in the Authentication object

Microservice architecture

Automated Testing

1. Create automated tests for each microservice

- Disable service registration and discovery

src/test/resources/application-test.properties

```
eureka.client.enabled=false
```

- Use an embedded database
- Mock/fake dependencies from other services

Microservice architecture

Automated Testing

2. Create end-to-end tests for the whole system

- Launch all services using containers
- Test using RestClient or another HTTP client

@Test

```
fun `POST to orders through gateway creates order and notifies kitchen`() {  
    val gatewayUrl = "http://localhost:${gateway.getMappedPort(8080)}"
```

gateway is a reference to
a container

```
    await().atMost(Duration.ofSeconds(30)).untilAsserted {
```

```
        val response = restClient.post()  
            .uri("$gatewayUrl/orders")  
            .body(...)   
            .retrieve()  
            .toEntity(Map::class.java)
```

have to wait until the gateway is
registered with the service registry

```
        assertThat(response.statusCode).isEqualTo(HttpStatus.CREATED)  
        assertThat(response.body!!["status"]).isEqualTo("PREPARING")
```

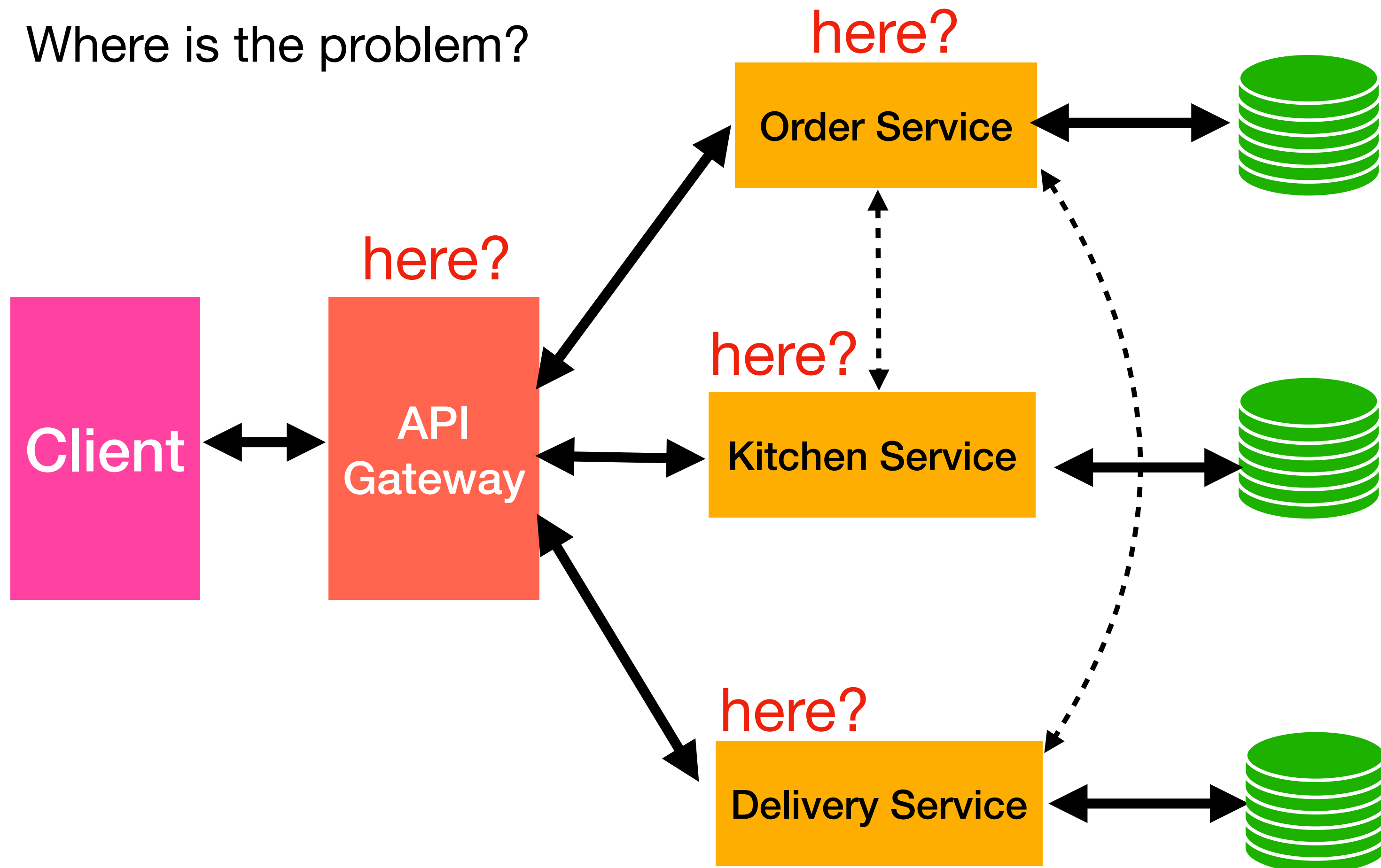
```
    }  
}
```

Microservice architecture

Observability

POST /orders returned 500

Where is the problem?



- Distributed systems are harder to debug
- Failures are partial and unpredictable
- Need to trace requests across multiple services

Microservice architecture

Observability

Traces - Request flow across services

```
[INFO] [trace_id=abc123xyz span_id=1] API Gateway - Received POST /orders
[INFO] [trace_id=abc123xyz span_id=1] API Gateway - Forwarding request to Order Service

[INFO] [trace_id=abc123xyz span_id=2 parent_span=1] Order Service - Creating order
[INFO] [trace_id=abc123xyz span_id=3 parent_span=2] Order DB - INSERT order (order_id=789) - SUCCESS

[INFO] [trace_id=abc123xyz span_id=2] Order Service - Calling Kitchen Service

[INFO] [trace_id=abc123xyz span_id=4 parent_span=2] Kitchen Service - Creating kitchen ticket
[INFO] [trace_id=abc123xyz span_id=5 parent_span=4] Kitchen DB - INSERT ticket (ticket_id=456) - SUCCESS

[INFO] [trace_id=abc123xyz span_id=4] Kitchen Service - Calling Delivery Service

[INFO] [trace_id=abc123xyz span_id=6 parent_span=4] Delivery Service - Scheduling delivery
[INFO] [trace_id=abc123xyz span_id=7 parent_span=6] Delivery DB - INSERT delivery (delivery_id=123) - SUCCESS

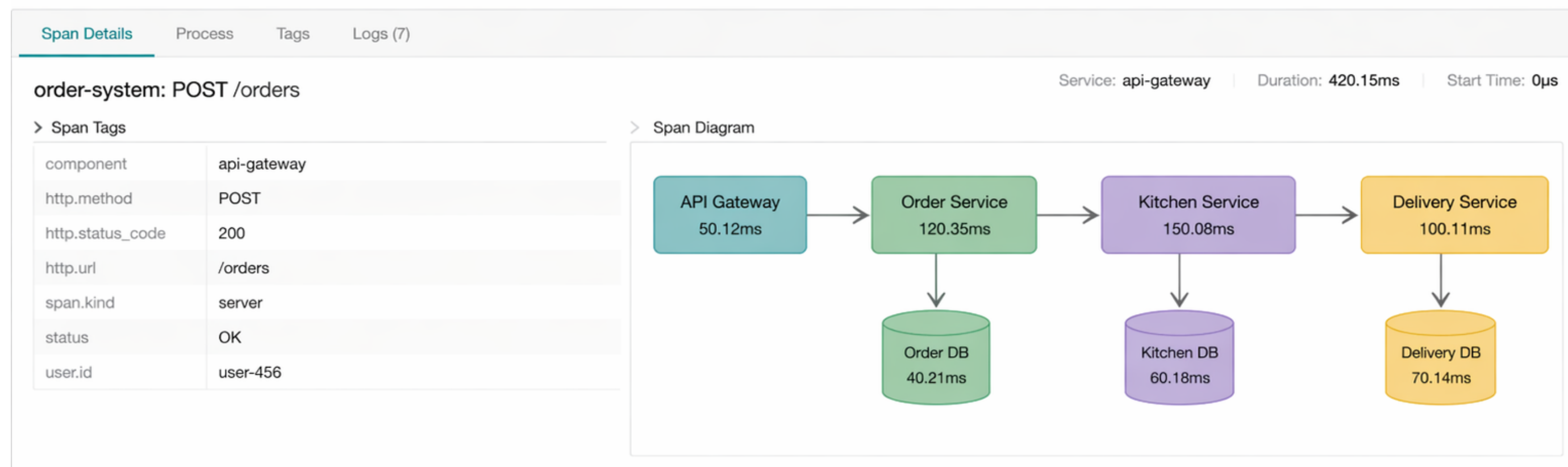
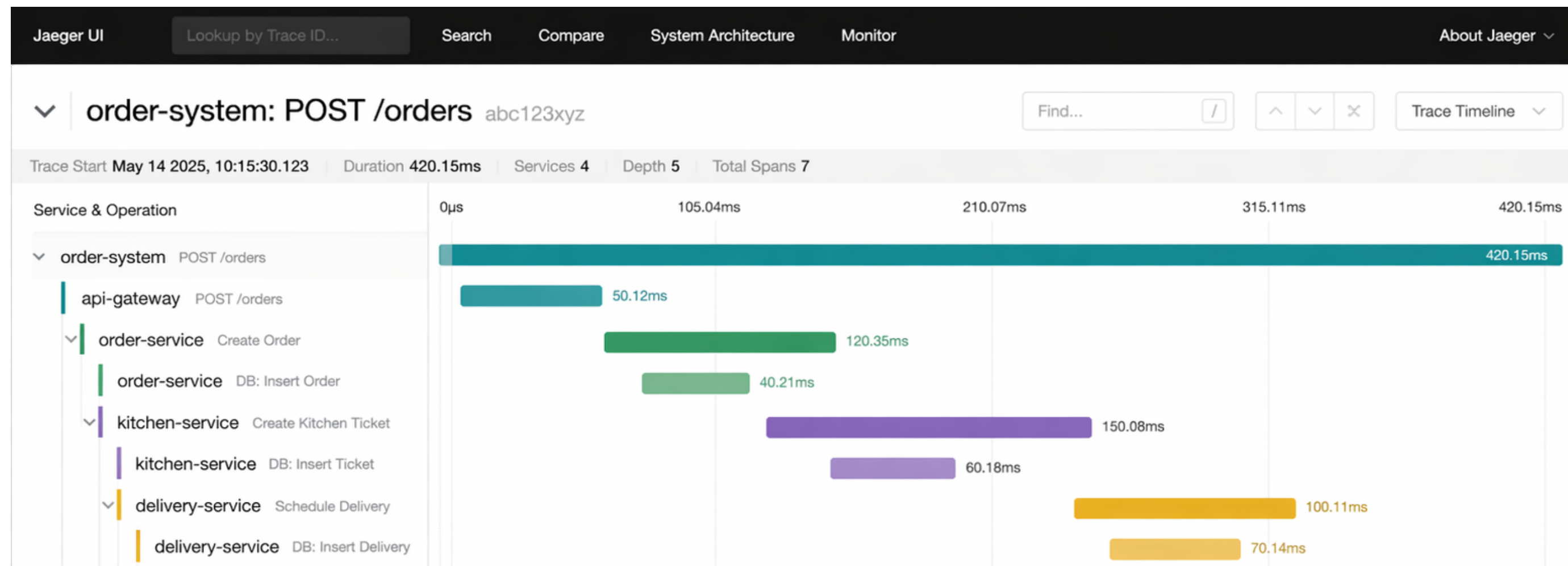
[INFO] [trace_id=abc123xyz span_id=6] Delivery Service - Delivery scheduled successfully
[INFO] [trace_id=abc123xyz span_id=4] Kitchen Service - Kitchen ticket created successfully
[INFO] [trace_id=abc123xyz span_id=2] Order Service - Order created successfully

[INFO] [trace_id=abc123xyz span_id=1] API Gateway - Response 200 OK (total_time=420ms)
```

Microservice architecture

Observability

Traces - Request flow across services



Example using Jaeger

<https://www.jaegertracing.io>

Microservice architecture

Observability

“Poor man’s” approach

For each microservice, log:

- All requests/response codes along with the authenticated user
Use an HandlerInterceptor
- All outgoing requests to other services
Use a Custom Feign Logger
- All queries to the database

traceability will come from a combination of the authenticated user and timestamps

Microservice architecture

Observability

Besides tracing, **observability** includes two other features:

- **Metrics** - Quantitative data (CPU usage, latency, error rates)
- **Logs** - Detailed event records (what happened & when)

Microservice architecture

Live demo

Live demo of a microservices architecture spring application

Midterm Test

- Midterm test will cover all material from lesson 1 through lesson 8 (inclusive).
- You may bring one A4 sheet (front and back) with any notes you choose. It must be handwritten by you - computer-printed notes or notes written by classmates are not allowed.
- You may not bring additional scratch paper.
- The exam must be answered directly on the test paper, using pen or pencil.
- There will be a multiple-choice section (worth at most half of the total grade). Incorrect answers will be penalized. Definitions may appear, but questions are more likely to focus on comparisons between approaches, best practices, etc. Small code snippets may also be included.
- There will be “open-ended” but objective questions (no long essays are expected). In these questions, you may be asked to define architectures (such as monoliths, microservices, API-based, or SSR), write or complete standard Spring code (such as controllers, services, entities, and repositories), or interpret/complete/draw information flow diagrams between system components.